ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

Отчет
о выполнении лабораторной работы № 3
"Многослойные сети. Алгоритм обратного распространения ошибки"
по дисциплине "Нейронные сети"

Вариант задания № 13

Выполнил студент
группы ЗФИбд-01-24
Князев А. М. _____

Проверил и принял
Леонов С. С. _____

с оценкой _____

Москва, 2026

Цель работы: исследование свойств многослойной нейронной сети прямого распространения и алгоритмов ее обучения, применение сети в задачах классификации и аппроксимации функции.

Основные этапы работы:
1. Использовать многослойную нейронную сеть для классификации точек в случае, когда классы не являются линейно разделимыми.
2. Использовать многослойную нейронную сеть для аппроксимации функции. Произвести обучение с помощью метода первого порядка (traingdx).
3. Использовать многослойную нейронную сеть для аппроксимации функции. Произвести обучение с помощью метода второго порядка (trainbfg).

Оборудование: персональный компьютер. **Программное обеспечение:** Python 3.x, Jupyter Notebook, библиотеки PyTorch, NumPy и Matplotlib.

Сценарий выполнения работы

Этап 1. Классификация линейно неразделимых классов

Заданы 3 линейно неразделимых класса. Точки, принадлежащие одному классу, лежат на алгебраической линии. Параметры кривых приведены в таблице 1.

Таблица 1 — Параметры кривых для генерации точек

Класс	Тип кривой	Параметры	Число точек
1	Эллипс (окружность)	$a = b = 0.3$, $x_0 = 0$, $y_0 = 0$	60
2	Эллипс (окружность)	$a = b = 0.7$, $x_0 = 0$, $y_0 = 0$	100

3	Парабола $y^2 = 2px$	$p = 1$, $x_0 = -0.8$, $y_0 = 0$	120

Для генерации точек использовались параметрические уравнения. Для эллипсов: $x = a \cdot \cos(t)$, $y = b \cdot \sin(t)$, $t \in [0, 2\pi]$. Для параболы: $x = t^2/2 + x_0$, $y = t$, $t \in [-2, 2]$. Из полученных множеств случайным образом (randperm) выбраны 60, 100 и 120 точек соответственно.

Множество точек каждого класса разделено на обучающее (70%), контрольное (20%) и тестовое (10%) подмножества с помощью функции dividerand. Соответствующие подмножества объединены в общую обучающую выборку.

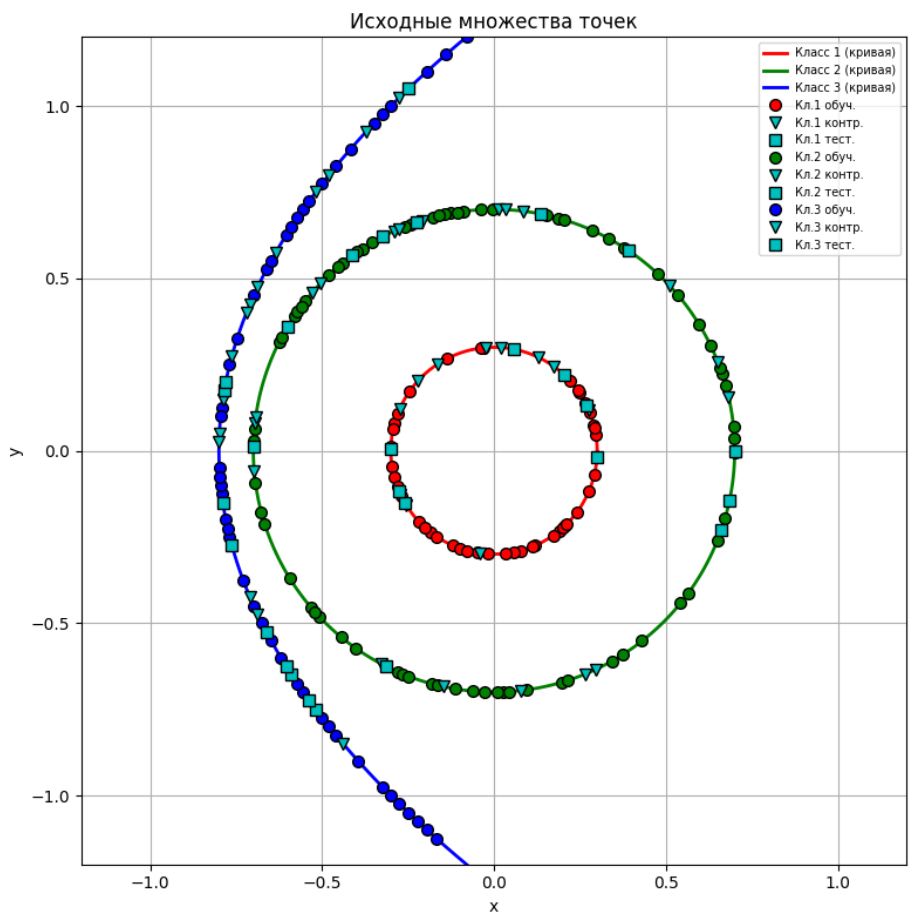

Рис. 1 — Исходные множества точек для трёх классов

Параметр	Значение
Тип сети	feedforwardnet (многослойная прямого распространения)
Архитектура	[2] → [20, tansig] → [3, sigmoid]
Алгоритм обучения	RProp (trainrp)
Число эпох (epochs)	1500
max_fail	1500
goal	1e-5
Число параметров	103

Таблица 2 — Структура сети (Этап 1)

Сеть создана с 20 нейронами в скрытом слое с функцией активации tansig (tanh) и 3 нейронами в выходном слое с функцией активации sigmoid. Входное множество лежит в диапазоне [−1.2, 1.2], выходное — в диапазоне [0, 1] по каждой из координат.

Весовые коэффициенты инициализированы с помощью функции по умолчанию (init). Обучение выполнено алгоритмом RProp.

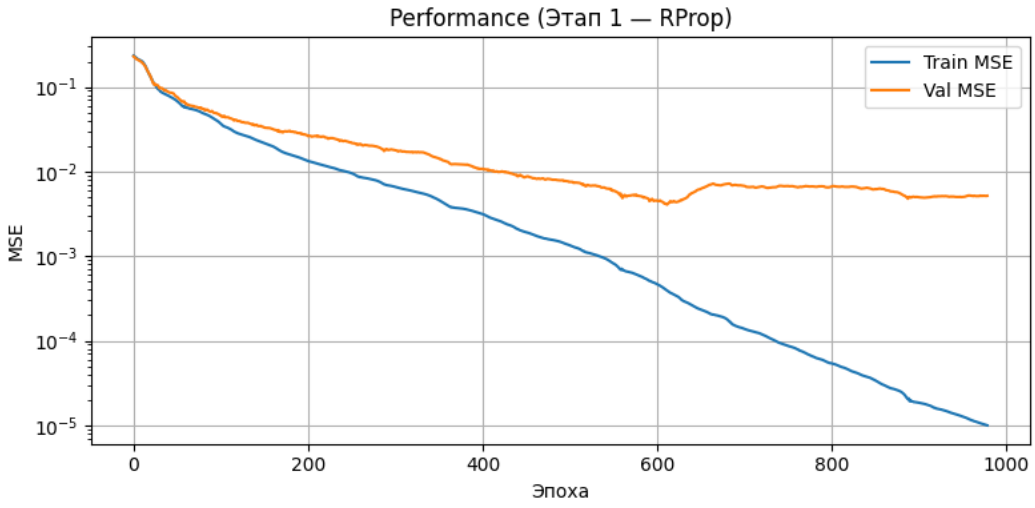

Рис. 2 — Performance (график обучения, Этап 1)

Результаты классификации:

Выход сети преобразован по правилу: oij = 1 если aij > 0.5, иначе 0.

Подмножество	Правильно	Всего	Точность
Обучающее	___	___	___%
Контрольное	___	___	___%
Тестовое	___	___	___%

Таблица 3 — Результаты классификации

Для классификации области [−1.2, 1.2] × [−1.2, 1.2] задана сетка с шагом h = 0.025. Выход сети для каждой точки определяет принадлежность к трём классам. Компоненты выходного вектора задают интенсивность цветов RGB: (1,0,0) — красный (класс 1), (0,1,0) — зелёный (класс 2), (0,0,1) — синий (класс 3).

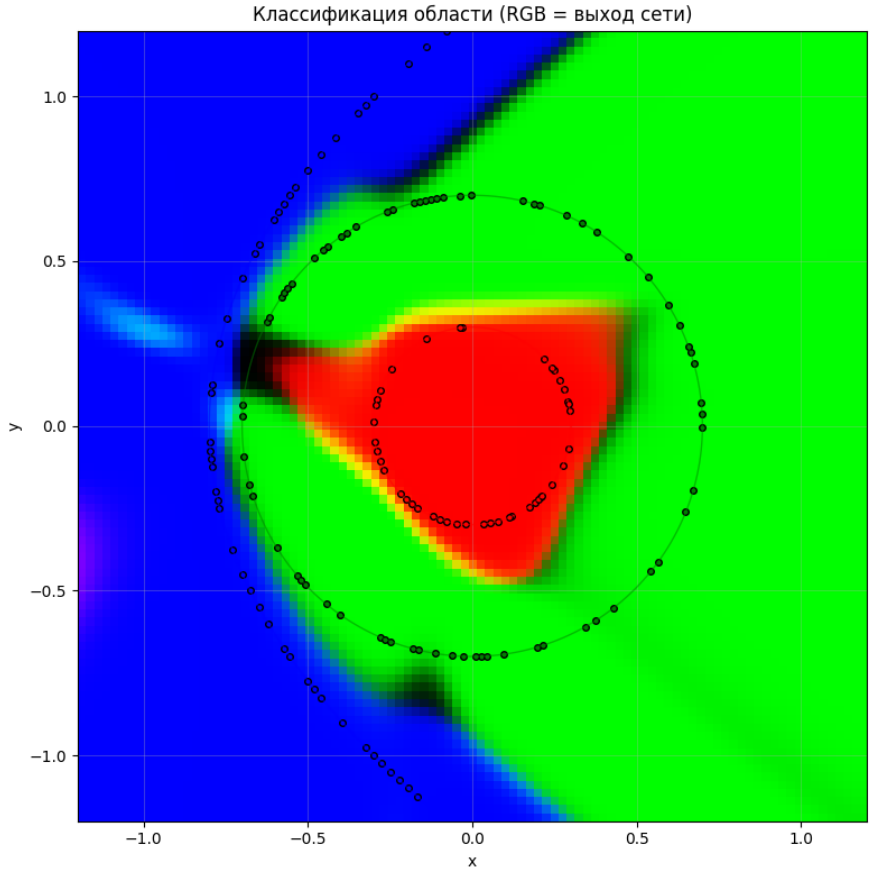

Рис. 3 — Классификация области (RGB = выход сети)

Этап 2. Аппроксимация функции (метод первого порядка)

Задан обучающий набор {t, x(t)}, где $x = \cos(t^2 - 2t + 3)$, $t \in [0, 5]$, шаг $h = 0.02$. Всего 251 точка.

С конца временной последовательности выделены 10% отсчётов на контрольное подмножество. Тестовое подмножество оставлено пустым.

Рис. 4 — Целевая функция $x = \cos(t^2 - 2t + 3)$

Параметр	Значение
Архитектура	[1] → [10, tansig] → [1, purelin]
Алгоритм обучения	traingdx (SGD + момент + адаптивный lr)
lr	0.05
mc (момент)	0.9
lr_inc	1.05
Число эпох (epochs)	2000
max_fail	600
goal	1e-8

Таблица 4 — Структура сети (Этап 2)

Метод traingdx реализует градиентный спуск с моментом и адаптивным шагом. При уменьшении ошибки скорость обучения увеличивается в lr_inc раз. При значительном увеличении ошибки (более 4%) веса откатываются, а скорость обучения уменьшается.

Обучение проводилось несколько раз с различными начальными весами. Выбран лучший результат по минимальной MSE на обучающем подмножестве.

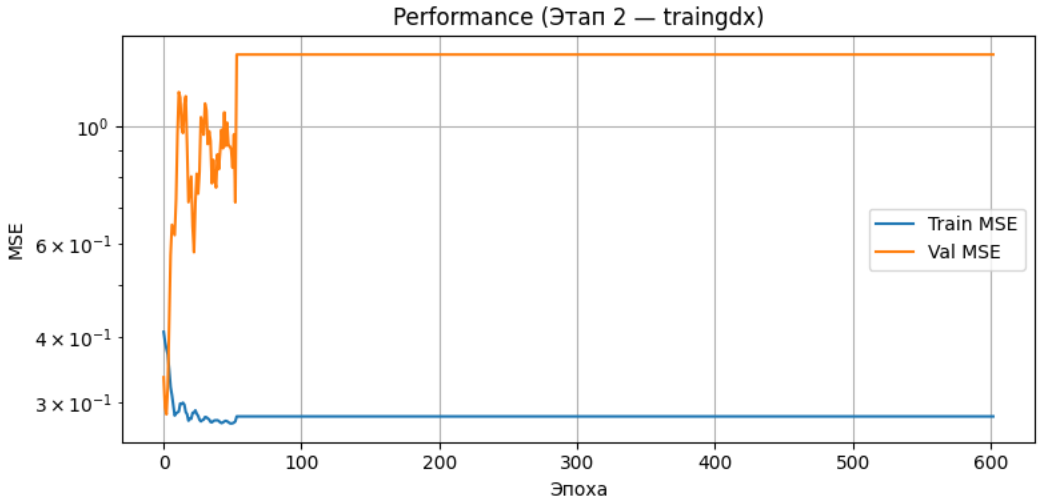

Рис. 5 — Performance (Этап 2 — traingdx)

Подмножество	MSE	MAE	Max|err|	R^2
Обучающее	3.701158e-01	4.775671e-01	1.308966e+00	0.089738
Контрольное	2.852418e-01	4.716484e-01	9.265240e-01	-0.140060

Таблица 5 — Показатели качества (Этап 2)

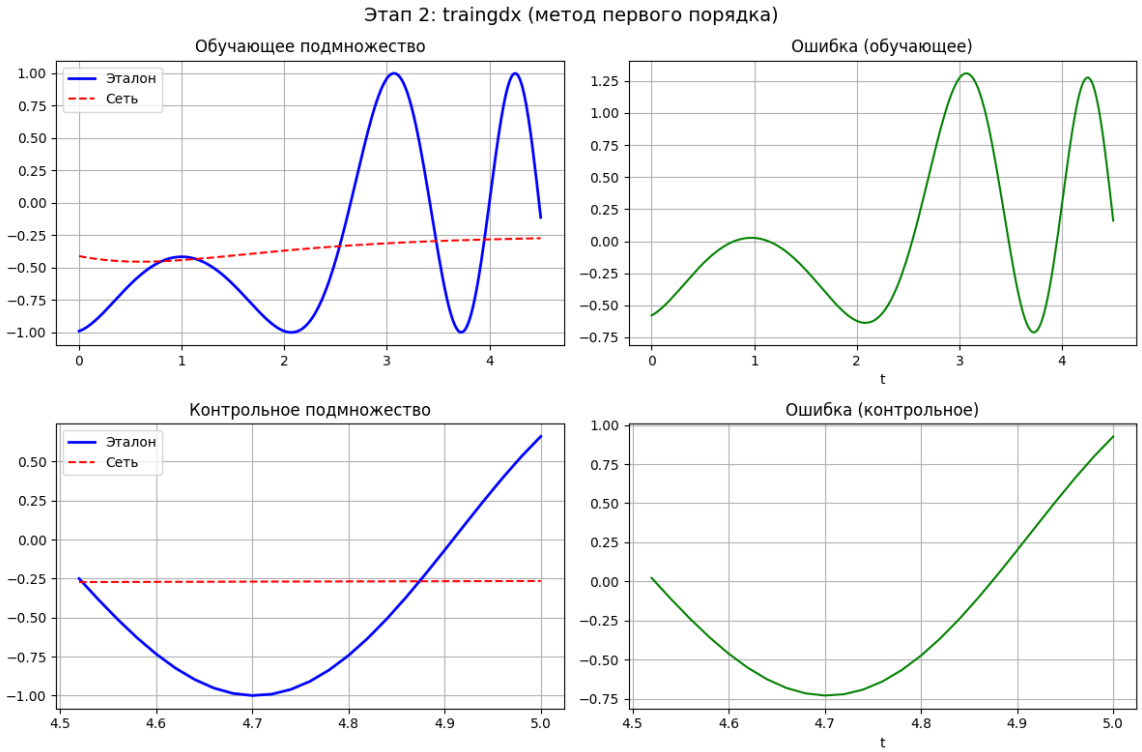

Рис. 6 — Аппроксимация и ошибка (Этап 2 — traingdx)

Этап 3. Аппроксимация функции (метод второго порядка)

Использована та же функция $x = \cos(t^2 - 2t + 3)$ и та же архитектура сети. Для обучения применён квазиньютоновский метод BFGS (trainbfg), реализованный через оптимизатор LBFGS.

Таблица 6 — Структура сети (Этап 3)

Параметр	Значение
Архитектура	[1] → [10, tansig] → [1, purelin]
Алгоритм обучения	trainbfg (LBFGS — квазиньютоновский)
Число эпох (epochs)	600
max_fail	600
goal	1e-8

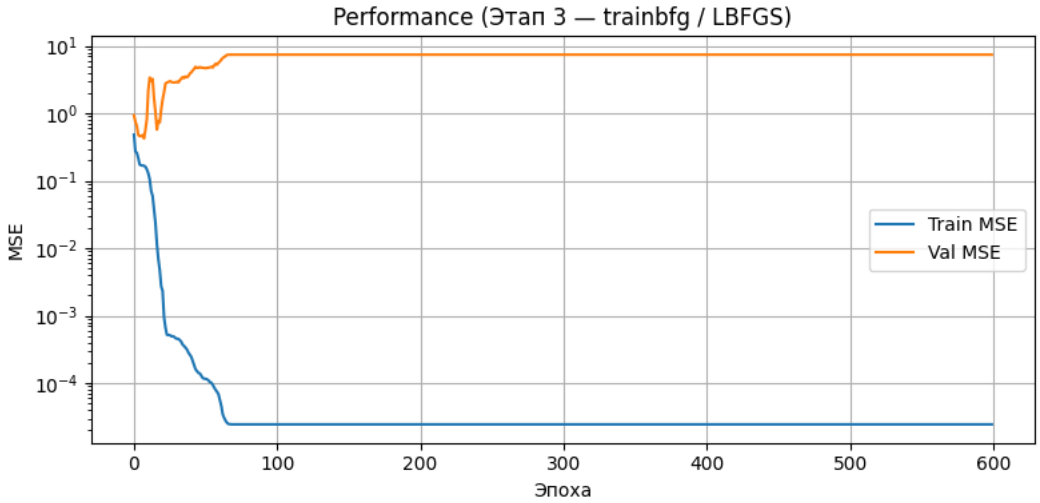

Рис. 7 — Performance (Этап 3 — trainbfg)

Подмножество	MSE	MAE	Max|err|	R^2
Обучающее	___	___	___	___
Контрольное	___	___	___	___

Таблица 7 — Показатели качества (Этап 3)

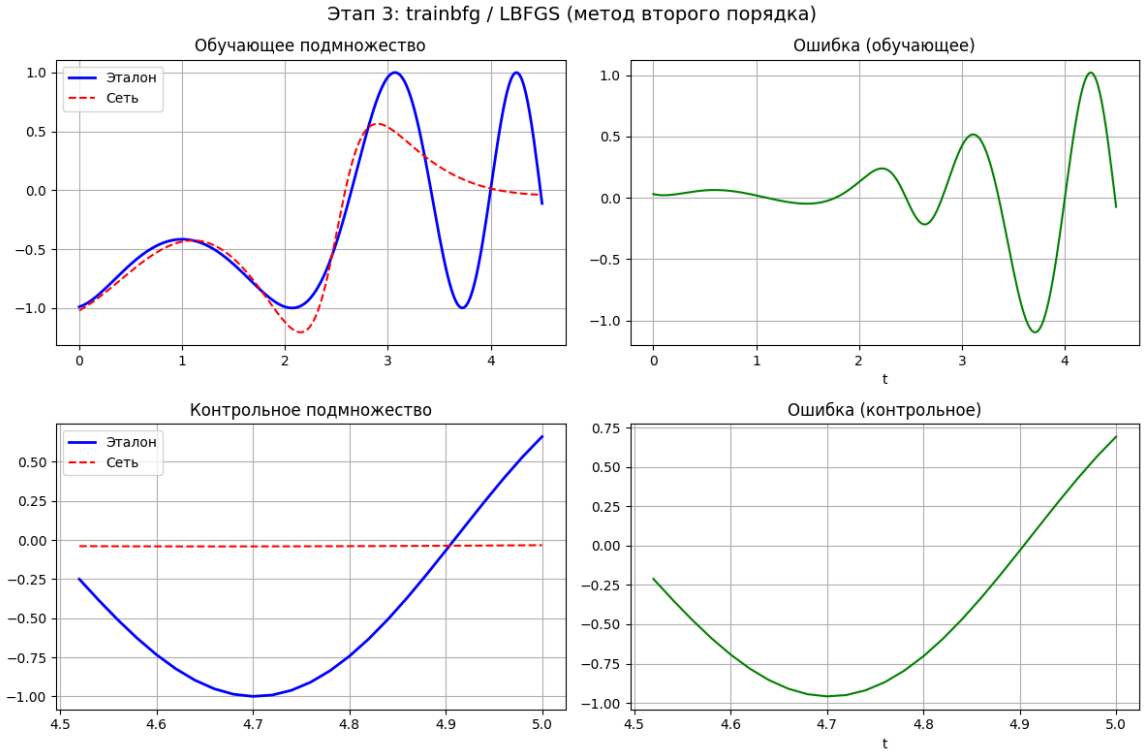

Рис. 8 — Аппроксимация и ошибка (Этап 3 — trainbfg)

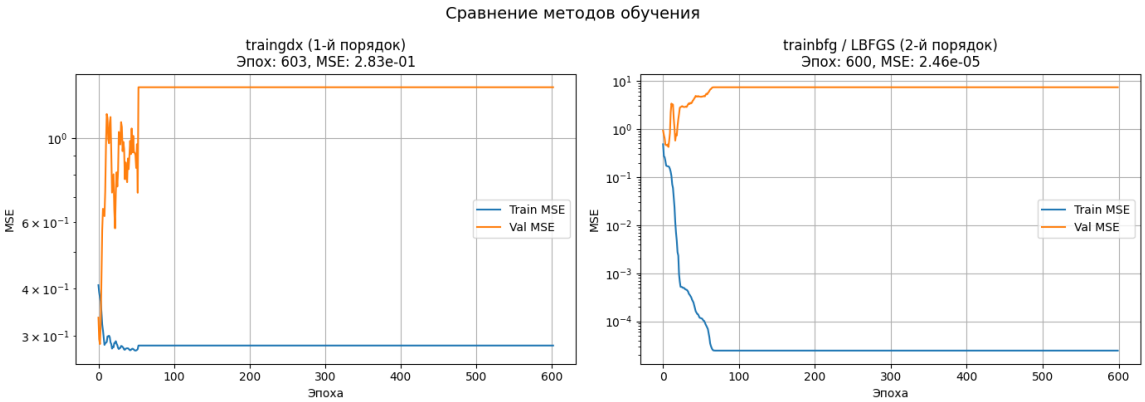

Рис. 9 — Сравнение методов обучения

Код программы

```
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.manual_seed(42)
np.random.seed(42)

t = np.arange(0, 2 * np.pi, 0.025)

# Класс 1: окружность r=0.3
x1_all = 0.3 * np.cos(t)
y1_all = 0.3 * np.sin(t)

# Класс 2: окружность r=0.7
x2_all = 0.7 * np.cos(t)
y2_all = 0.7 * np.sin(t)

# Класс 3: парабола y^2 = 2px, p=1, x0=-0.8, y0=0
# Параметрическая форма: x = t^2/2 + x0, y = t + y0
t_par = np.arange(-2.0, 2.0, 0.025)
x3_all = t_par**2 / 2.0 - 0.8
y3_all = t_par.copy()

# Случайный выбор точек (randperm)
idx1 = np.random.permutation(len(t))[:60]
idx2 = np.random.permutation(len(t))[:100]
idx3 = np.random.permutation(len(t_par))[:120]

c1_x, c1_y = x1_all[idx1], y1_all[idx1]
c2_x, c2_y = x2_all[idx2], y2_all[idx2]
c3_x, c3_y = x3_all[idx3], y3_all[idx3]

print(f"Класс 1 (эллипс r=0.3): {len(c1_x)} точек")
print(f"Класс 2 (эллипс r=0.7): {len(c2_x)} точек")
print(f"Класс 3 (парабола):     {len(c3_x)} точек")

def divide_rand(x, y, train_r=0.7, val_r=0.2):
    n = len(x)
    perm = np.random.permutation(n)
    n_tr = int(n * train_r)
    n_val = int(n * (train_r + val_r))
```

```
    return (x[perm[:n_tr]], y[perm[:n_tr]],
            x[perm[n_tr:n_val]], y[perm[n_tr:n_val]],
            x[perm[n_val:]], y[perm[n_val:]])

c1_tr_x, c1_tr_y, c1_vl_x, c1_vl_y, c1_ts_x, c1_ts_y =
divide_rand(c1_x, c1_y)
c2_tr_x, c2_tr_y, c2_vl_x, c2_vl_y, c2_ts_x, c2_ts_y =
divide_rand(c2_x, c2_y)
c3_tr_x, c3_tr_y, c3_vl_x, c3_vl_y, c3_ts_x, c3_ts_y =
divide_rand(c3_x, c3_y)

for name, tr, vl, ts in [
    ("Класс 1", c1_tr_x, c1_vl_x, c1_ts_x),
    ("Класс 2", c2_tr_x, c2_vl_x, c2_ts_x),
    ("Класс 3", c3_tr_x, c3_vl_x, c3_ts_x),
]:
    print(f"{name}: train={len(tr)}, val={len(vl)}, test={len(ts)}")

fig, ax = plt.subplots(figsize=(8, 8))

# Исходные кривые
ax.plot(x1_all, y1_all, '-r', linewidth=2, label='Класс 1 (кривая)')
ax.plot(x2_all, y2_all, '-g', linewidth=2, label='Класс 2 (кривая)')
ax.plot(x3_all, y3_all, '-b', linewidth=2, label='Класс 3 (кривая)')

# Класс 1: обучающее, контрольное, тестовое
ax.plot(c1_tr_x, c1_tr_y, 'o', color='r', markeredgecolor='k',
        markerfacecolor='r', markersize=7, label='Кл.1 обуч.')
ax.plot(c1_vl_x, c1_vl_y, 'v', color='r', markeredgecolor='k',
        markerfacecolor='c', markersize=7, label='Кл.1 контр.')
ax.plot(c1_ts_x, c1_ts_y, 's', color='r', markeredgecolor='k',
        markerfacecolor='c', markersize=7, label='Кл.1 тест.')

# Класс 2
ax.plot(c2_tr_x, c2_tr_y, 'o', color='g', markeredgecolor='k',
        markerfacecolor='g', markersize=7, label='Кл.2 обуч.')
ax.plot(c2_vl_x, c2_vl_y, 'v', color='g', markeredgecolor='k',
        markerfacecolor='c', markersize=7, label='Кл.2 контр.')
ax.plot(c2_ts_x, c2_ts_y, 's', color='g', markeredgecolor='k',
        markerfacecolor='c', markersize=7, label='Кл.2 тест.')

# Класс 3
ax.plot(c3_tr_x, c3_tr_y, 'o', color='b', markeredgecolor='k',
        markerfacecolor='b', markersize=7, label='Кл.3 обуч.')
ax.plot(c3_vl_x, c3_vl_y, 'v', color='b', markeredgecolor='k',
        markerfacecolor='c', markersize=7, label='Кл.3 контр.')
```

```
ax.plot(c3_ts_x, c3_ts_y, 's', color='b', markeredgecolor='k',
        markerfacecolor='c', markersize=7, label='Кл.3 тест.')


ax.set_xlim(-1.2, 1.2)
ax.set_ylim(-1.2, 1.2)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Исходные множества точек')
ax.legend(fontsize=7, loc='upper right')
ax.grid(True)
ax.set_aspect('equal')
plt.tight_layout()
plt.show()


# Объединяем подмножества каждого класса
def combine_sets(*classes_data):
    """classes_data: [(tr_x, tr_y, vl_x, vl_y, ts_x, ts_y), ...]"""
    X_tr = np.column_stack([np.concatenate([c[0] for c in
classes_data]),
                            np.concatenate([c[1] for c in
classes_data])])
    X_vl = np.column_stack([np.concatenate([c[2] for c in
classes_data]),
                            np.concatenate([c[3] for c in
classes_data])])
    X_ts = np.column_stack([np.concatenate([c[4] for c in
classes_data]),
                            np.concatenate([c[5] for c in
classes_data])])
    return X_tr, X_vl, X_ts


def make_targets(sizes, n_classes=3):
    rows = []
    for i, n in enumerate(sizes):
        one_hot = np.zeros((n, n_classes))
        one_hot[:, i] = 1.0
        rows.append(one_hot)
    return np.vstack(rows)


cls1 = (c1_tr_x, c1_tr_y, c1_vl_x, c1_vl_y, c1_ts_x, c1_ts_y)
cls2 = (c2_tr_x, c2_tr_y, c2_vl_x, c2_vl_y, c2_ts_x, c2_ts_y)
cls3 = (c3_tr_x, c3_tr_y, c3_vl_x, c3_vl_y, c3_ts_x, c3_ts_y)


X_tr, X_vl, X_ts = combine_sets(cls1, cls2, cls3)


Y_tr = make_targets([len(c1_tr_x), len(c2_tr_x), len(c3_tr_x)])
Y_vl = make_targets([len(c1_vl_x), len(c2_vl_x), len(c3_vl_x)])
```

```
Y_ts = make_targets([len(c1_ts_x), len(c2_ts_x), len(c3_ts_x)])

# divideind — индексы для разделения
trnInd = len(X_tr)
tstInd = trnInd + len(X_vl)
proInd = tstInd + len(X_ts)

# Объединяем всё в единую обучающую выборку
X_all = np.vstack([X_tr, X_vl, X_ts])
Y_all = np.vstack([Y_tr, Y_vl, Y_ts])

# Конвертация в тензоры
X_tr_t = torch.FloatTensor(X_tr)
Y_tr_t = torch.FloatTensor(Y_tr)
X_vl_t = torch.FloatTensor(X_vl)
Y_vl_t = torch.FloatTensor(Y_vl)
X_ts_t = torch.FloatTensor(X_ts)
Y_ts_t = torch.FloatTensor(Y_ts)

print(f"Обучающее:   X={X_tr.shape}, Y={Y_tr.shape}")
print(f"Контрольное: X={X_vl.shape}, Y={Y_vl.shape}")
print(f"Тестовое:    X={X_ts.shape}, Y={Y_ts.shape}")
print(f"\ndivideInd: trainInd=1:{trnInd}, valInd={trnInd+1}:{tstInd},
testInd={tstInd+1}:{proInd}")

class ClassificationNet(nn.Module):
    def __init__(self, hidden_size=20):
        super().__init__()
        self.hidden = nn.Linear(2, hidden_size)
        self.output = nn.Linear(hidden_size, 3)

    def forward(self, x):
        x = torch.tanh(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x

model_cls = ClassificationNet(20)
print(model_cls)
print(f"\nОбщее число параметров: {sum(p.numel() for p in
model_cls.parameters())}")

def train_model(model, optimizer, X_tr, Y_tr, X_vl, Y_vl,
                epochs=1500, max_fail=1500, goal=1e-5,
                use_lbfgs=False, verbose_step=200):
    """Обучение с ранней остановкой по контрольному подмножеству."""
    criterion = nn.MSELoss()
```

```
    best_val = float('inf')
    fails = 0
    best_state = {k: v.clone() for k, v in model.state_dict().items()}
    hist_tr, hist_vl = [], []

    for ep in range(1, epochs + 1):
        model.train()
        if use_lbfgs:
            def closure():
                optimizer.zero_grad()
                loss = criterion(model(X_tr), Y_tr)
                loss.backward()
                return loss
            tr_loss = optimizer.step(closure).item()
        else:
            optimizer.zero_grad()
            loss = criterion(model(X_tr), Y_tr)
            loss.backward()
            optimizer.step()
            tr_loss = loss.item()

        model.eval()
        with torch.no_grad():
            vl_loss = criterion(model(X_vl), Y_vl).item()

        hist_tr.append(tr_loss)
        hist_vl.append(vl_loss)

        if vl_loss < best_val:
            best_val = vl_loss
            fails = 0
            best_state = {k: v.clone() for k, v in
model.state_dict().items()}
        else:
            fails += 1

        if tr_loss <= goal:
            print(f"Эпоха {ep}: цель достигнута, MSE = {tr_loss:.2e}")
            break
        if fails >= max_fail:
            print(f"Эпоха {ep}: ранняя остановка
(max_fail={max_fail})")
            break
        if ep % verbose_step == 0:
            print(f"Эпоха {ep}: train={tr_loss:.6f},
val={vl_loss:.6f}")
```

```
    if ep == epochs and tr_loss > goal and fails < max_fail:
        print(f"Эпоха {ep}: обучение завершено,
train_MSE={tr_loss:.6e}")

    model.load_state_dict(best_state)
    return hist_tr, hist_vl

torch.manual_seed(42)
model_cls = ClassificationNet(20)
optimizer_cls = torch.optim.Rprop(model_cls.parameters())

hist_tr1, hist_vl1 = train_model(
    model_cls, optimizer_cls,
    X_tr_t, Y_tr_t, X_vl_t, Y_vl_t,
    epochs=1500, max_fail=1500, goal=1e-5
)

# График обучения (Performance)
fig, ax = plt.subplots(figsize=(8, 4))
ax.semilogy(hist_tr1, label='Train MSE')
ax.semilogy(hist_vl1, label='Val MSE')
ax.set_xlabel('Эпоха')
ax.set_ylabel('MSE')
ax.set_title('Performance (Этап 1 — RProp)')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()

print("=== Структура сети (Этап 1) ===")
print(f"Тип сети:            feedforwardnet (многослойная прямого
распространения)")
print(f"Архитектура:         [2] -> [20, tansig] -> [3, sigmoid]")
print(f"Алгоритм обучения:   RProp (trainrp)")
print(f"Параметры обучения:  epochs=1500, max_fail=1500, goal=1e-5")
print(f"Всего эпох обучения: {len(hist_tr1)}")
print(f"Финальная MSE train: {hist_tr1[-1]:.6e}")
print(f"Финальная MSE val:   {hist_vl1[-1]:.6e}")
print()
for name, param in model_cls.named_parameters():
    print(f"{name}: shape={list(param.shape)}")
    print(param.data.numpy().round(4))
    print()

def evaluate_cls(model, X, Y, name=""):
    model.eval()
    with torch.no_grad():
```

```
        out = model(X)
    pred = (out > 0.5).float()
    correct = (pred == Y).all(dim=1).sum().item()
    total = Y.shape[0]
    print(f"{name}: {correct}/{total} правильно классифицировано
({100*correct/total:.1f}%)")
    return pred


print("--- Результаты классификации ---")
pred_tr = evaluate_cls(model_cls, X_tr_t, Y_tr_t, "Обучающее")
pred_vl = evaluate_cls(model_cls, X_vl_t, Y_vl_t, "Контрольное")
pred_ts = evaluate_cls(model_cls, X_ts_t, Y_ts_t, "Тестовое")


h = 0.025
grid_vals = np.arange(-1.2, 1.2 + h, h)
gx, gy = np.meshgrid(grid_vals, grid_vals)
grid_pts = torch.FloatTensor(np.column_stack([gx.ravel(), gy.ravel()]))


model_cls.eval()
with torch.no_grad():
    grid_out = model_cls(grid_pts).numpy()


# Округляем до десятых для формирования таблицы цветов
grid_rounded = np.round(grid_out, 1)
# Уникальные цвета (colormap)
unique_colors = np.unique(grid_rounded, axis=0)
print(f"Таблица цветов ({len(unique_colors)} уникальных):")
for i, c in enumerate(unique_colors):
    print(f"  {i}: RGB = {c}")


# Формируем RGB-изображение из выхода сети
grid_rgb = np.clip(grid_out, 0, 1).reshape(gx.shape[0], gx.shape[1], 3)


fig, ax = plt.subplots(figsize=(8, 8))
ax.imshow(grid_rgb, extent=[-1.2, 1.2, -1.2, 1.2], origin='lower',
aspect='equal')


# Наложение точек обучающей выборки
ax.plot(c1_tr_x, c1_tr_y, 'o', markeredgecolor='k',
markerfacecolor='r', markersize=4)
ax.plot(c2_tr_x, c2_tr_y, 'o', markeredgecolor='k',
markerfacecolor='g', markersize=4)
ax.plot(c3_tr_x, c3_tr_y, 'o', markeredgecolor='k',
markerfacecolor='b', markersize=4)


# Исходные кривые
```

```
ax.plot(x1_all, y1_all, 'r-', linewidth=1, alpha=0.5)
ax.plot(x2_all, y2_all, 'g-', linewidth=1, alpha=0.5)
ax.plot(x3_all, y3_all, 'b-', linewidth=1, alpha=0.5)

ax.set_xlim(-1.2, 1.2)
ax.set_ylim(-1.2, 1.2)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Классификация области (RGB = выход сети)')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

t_func = np.arange(0, 5 + 0.02, 0.02)
x_func = np.cos(t_func**2 - 2 * t_func + 3)

n_total = len(t_func)
n_val_f = int(n_total * 0.1)   # 10% с конца — контрольное
n_train_f = n_total - n_val_f

# Разделение: первые 90% — обучающее, последние 10% — контрольное
t_tr_f, x_tr_f = t_func[:n_train_f], x_func[:n_train_f]
t_vl_f, x_vl_f = t_func[n_train_f:], x_func[n_train_f:]

print(f"Всего точек: {n_total}")
print(f"Обучающее: {n_train_f}, контрольное: {n_val_f}")

# Конвертация в тензоры
T_tr_f = torch.FloatTensor(t_tr_f).unsqueeze(1)
X_tr_f = torch.FloatTensor(x_tr_f).unsqueeze(1)
T_vl_f = torch.FloatTensor(t_vl_f).unsqueeze(1)
X_vl_f = torch.FloatTensor(x_vl_f).unsqueeze(1)

# График целевой функции
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(t_func, x_func, 'b-', linewidth=2, label='x = cos(t² - 2t +
3)')
ax.axvline(t_func[n_train_f], color='r', linestyle='--', label='Граница
train/val')
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.set_title('Целевая функция')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()
```

```
class ApproxNet(nn.Module):
    def __init__(self, hidden_size=10):
        super().__init__()
        self.hidden = nn.Linear(1, hidden_size)
        self.output = nn.Linear(hidden_size, 1)

    def forward(self, x):
        x = torch.tanh(self.hidden(x))
        x = self.output(x)  # purelin — линейный выход
        return x



def train_gdx(model, X_tr, Y_tr, X_vl, Y_vl,
              lr=0.05, momentum=0.9, lr_inc=1.05, lr_dec=0.7,
              epochs=600, max_fail=600, goal=1e-8, verbose_step=100):
    """Обучение методом градиентного спуска с адаптивным шагом и
моментом (traingdx)."""
    criterion = nn.MSELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=lr,
momentum=momentum)

    best_val = float('inf')
    fails = 0
    best_state = {k: v.clone() for k, v in model.state_dict().items()}
    prev_state = {k: v.clone() for k, v in model.state_dict().items()}
    prev_loss = float('inf')
    hist_tr, hist_vl = [], []

    for ep in range(1, epochs + 1):
        prev_state = {k: v.clone() for k, v in
model.state_dict().items()}

        model.train()
        optimizer.zero_grad()
        loss = criterion(model(X_tr), Y_tr)
        loss.backward()
        optimizer.step()
        tr_loss = loss.item()

        # Адаптивный шаг обучения
        if tr_loss > prev_loss * 1.04:
            # Ошибка выросла — откат весов, уменьшение lr
            model.load_state_dict(prev_state)
            for pg in optimizer.param_groups:
                pg['lr'] = max(pg['lr'] * lr_dec, 1e-7)
```

```
            tr_loss = prev_loss
        elif tr_loss < prev_loss:
            for pg in optimizer.param_groups:
                pg['lr'] = min(pg['lr'] * lr_inc, 1.0)
        prev_loss = tr_loss

        model.eval()
        with torch.no_grad():
            vl_loss = criterion(model(X_vl), Y_vl).item()

        hist_tr.append(tr_loss)
        hist_vl.append(vl_loss)

        if vl_loss < best_val:
            best_val = vl_loss
            fails = 0
            best_state = {k: v.clone() for k, v in
model.state_dict().items()}
        else:
            fails += 1

        if tr_loss <= goal:
            print(f"Эпоха {ep}: цель достигнута, MSE = {tr_loss:.2e}")
            break
        if fails >= max_fail:
            print(f"Эпоха {ep}: ранняя остановка
(max_fail={max_fail})")
            break
        if ep % verbose_step == 0:
            cur_lr = optimizer.param_groups[0]['lr']
            print(f"Эпоха {ep}: train={tr_loss:.6f}, val={vl_loss:.6f},
lr={cur_lr:.6f}")

    if ep == epochs and tr_loss > goal and fails < max_fail:
        print(f"Эпоха {ep}: обучение завершено,
train_MSE={tr_loss:.6e}")

    model.load_state_dict(best_state)
    return hist_tr, hist_vl

# Обучение несколько раз, выбираем лучший результат
best_loss_gdx = float('inf')
best_state_gdx = None
best_hist_gdx = None

for seed in range(10):
```

```
    torch.manual_seed(seed)
    m = ApproxNet(10)
    h_tr, h_vl = train_gdx(m, T_tr_f, X_tr_f, T_vl_f, X_vl_f,
                           lr=0.05, momentum=0.9, lr_inc=1.05,
                           epochs=2000, max_fail=600, goal=1e-8,
                           verbose_step=9999)
    final = h_tr[-1]
    print(f"  seed={seed}: final_train_MSE = {final:.6e}")
    if final < best_loss_gdx:
        best_loss_gdx = final
        best_state_gdx = {k: v.clone() for k, v in
m.state_dict().items()}
        best_hist_gdx = (h_tr, h_vl)

model_gdx = ApproxNet(10)
model_gdx.load_state_dict(best_state_gdx)
hist_tr2, hist_vl2 = best_hist_gdx
print(f"\nЛучший результат: train_MSE = {best_loss_gdx:.6e}")

print("=== Структура сети (Этап 2) ===")
print(f"Архитектура:         [1] -> [10, tansig] -> [1, purelin]")
print(f"Алгоритм обучения:   traingdx (SGD + момент + адаптивный lr)")
print(f"Параметры:           lr=0.05, mc=0.9, lr_inc=1.05")
print(f"Эпох обучения:       {len(hist_tr2)}")
print(f"Финальная MSE train: {hist_tr2[-1]:.6e}")
print(f"Финальная MSE val:   {hist_vl2[-1]:.6e}")
print()
print("--- Весовые коэффициенты ---")
for name, param in model_gdx.named_parameters():
    print(f"{name}: shape={list(param.shape)}")
    print(param.data.numpy().round(4))
    print()

# Performance
fig, ax = plt.subplots(figsize=(8, 4))
ax.semilogy(hist_tr2, label='Train MSE')
ax.semilogy(hist_vl2, label='Val MSE')
ax.set_xlabel('Эпоха')
ax.set_ylabel('MSE')
ax.set_title('Performance (Этап 2 — traingdx)')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()

def evaluate_approx(model, T_data, X_data, name=""):
    model.eval()
    with torch.no_grad():
```

```
        pred = model(T_data).numpy().flatten()
    target = X_data.numpy().flatten()
    err = target - pred
    mse = np.mean(err**2)
    mae = np.mean(np.abs(err))
    max_err = np.max(np.abs(err))
    ss_res = np.sum(err**2)
    ss_tot = np.sum((target - target.mean())**2)
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
    print(f"{name}:")
    print(f"  MSE = {mse:.6e}")
    print(f"  MAE = {mae:.6e}")
    print(f"  Max|err| = {max_err:.6e}")
    print(f"  R²  = {r2:.6f}")
    return pred, err

# Обучающее подмножество
pred_tr2, err_tr2 = evaluate_approx(model_gdx, T_tr_f, X_tr_f,
"Обучающее")
print()
# Контрольное подмножество
pred_vl2, err_vl2 = evaluate_approx(model_gdx, T_vl_f, X_vl_f,
"Контрольное")

# Графики
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

axes[0, 0].plot(t_tr_f, x_tr_f, 'b-', linewidth=2, label='Эталон')
axes[0, 0].plot(t_tr_f, pred_tr2, 'r--', linewidth=1.5, label='Сеть')
axes[0, 0].set_title('Обучающее подмножество')
axes[0, 0].legend()
axes[0, 0].grid(True)

axes[0, 1].plot(t_tr_f, err_tr2, 'g-')
axes[0, 1].set_title('Ошибка (обучающее)')
axes[0, 1].set_xlabel('t')
axes[0, 1].grid(True)

axes[1, 0].plot(t_vl_f, x_vl_f, 'b-', linewidth=2, label='Эталон')
axes[1, 0].plot(t_vl_f, pred_vl2, 'r--', linewidth=1.5, label='Сеть')
axes[1, 0].set_title('Контрольное подмножество')
axes[1, 0].legend()
axes[1, 0].grid(True)

axes[1, 1].plot(t_vl_f, err_vl2, 'g-')
axes[1, 1].set_title('Ошибка (контрольное)')
axes[1, 1].set_xlabel('t')
```

```
axes[1, 1].grid(True)

plt.suptitle('Этап 2: traingdx (метод первого порядка)', fontsize=14)
plt.tight_layout()
plt.show()

# Обучение несколько раз, выбираем лучший результат
best_loss_bfg = float('inf')
best_state_bfg = None
best_hist_bfg = None

for seed in range(5):
    torch.manual_seed(seed)
    m = ApproxNet(10)
    opt = torch.optim.LBFGS(m.parameters(), lr=0.1,
                            max_iter=20,
line_search_fn='strong_wolfe')
    h_tr, h_vl = train_model(
        m, opt, T_tr_f, X_tr_f, T_vl_f, X_vl_f,
        epochs=600, max_fail=600, goal=1e-8,
        use_lbfgs=True, verbose_step=9999
    )
    final = h_tr[-1]
    print(f"  seed={seed}: final_train_MSE = {final:.6e}")
    if final < best_loss_bfg:
        best_loss_bfg = final
        best_state_bfg = {k: v.clone() for k, v in
m.state_dict().items()}
        best_hist_bfg = (h_tr, h_vl)

model_bfg = ApproxNet(10)
model_bfg.load_state_dict(best_state_bfg)
hist_tr3, hist_vl3 = best_hist_bfg
print(f"\nЛучший результат: train_MSE = {best_loss_bfg:.6e}")

print("=== Структура сети (Этап 3) ===")
print(f"Архитектура:         [1] -> [10, tansig] -> [1, purelin]")
print(f"Алгоритм обучения:   trainbfg (LBFGS — квазиньютоновский
метод)")
print(f"Эпох обучения:       {len(hist_tr3)}")
print(f"Финальная MSE train: {hist_tr3[-1]:.6e}")
print(f"Финальная MSE val:   {hist_vl3[-1]:.6e}")
print()
print("--- Весовые коэффициенты ---")
for name, param in model_bfg.named_parameters():
    print(f"{name}: shape={list(param.shape)}")
    print(param.data.numpy().round(4))
```

```
    print()

# Performance
fig, ax = plt.subplots(figsize=(8, 4))
ax.semilogy(hist_tr3, label='Train MSE')
ax.semilogy(hist_vl3, label='Val MSE')
ax.set_xlabel('Эпоха')
ax.set_ylabel('MSE')
ax.set_title('Performance (Этап 3 — trainbfg / LBFGS)')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()

# Обучающее подмножество
pred_tr3, err_tr3 = evaluate_approx(model_bfg, T_tr_f, X_tr_f,
"Обучающее")
print()
# Контрольное подмножество
pred_vl3, err_vl3 = evaluate_approx(model_bfg, T_vl_f, X_vl_f,
"Контрольное")

# Графики
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

axes[0, 0].plot(t_tr_f, x_tr_f, 'b-', linewidth=2, label='Эталон')
axes[0, 0].plot(t_tr_f, pred_tr3, 'r--', linewidth=1.5, label='Сеть')
axes[0, 0].set_title('Обучающее подмножество')
axes[0, 0].legend()
axes[0, 0].grid(True)

axes[0, 1].plot(t_tr_f, err_tr3, 'g-')
axes[0, 1].set_title('Ошибка (обучающее)')
axes[0, 1].set_xlabel('t')
axes[0, 1].grid(True)

axes[1, 0].plot(t_vl_f, x_vl_f, 'b-', linewidth=2, label='Эталон')
axes[1, 0].plot(t_vl_f, pred_vl3, 'r--', linewidth=1.5, label='Сеть')
axes[1, 0].set_title('Контрольное подмножество')
axes[1, 0].legend()
axes[1, 0].grid(True)

axes[1, 1].plot(t_vl_f, err_vl3, 'g-')
axes[1, 1].set_title('Ошибка (контрольное)')
axes[1, 1].set_xlabel('t')
axes[1, 1].grid(True)
```

```
plt.suptitle('Этап 3: trainbfg / LBFGS (метод второго порядка)',
fontsize=14)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Этап 2
axes[0].semilogy(hist_tr2, label='Train MSE')
axes[0].semilogy(hist_vl2, label='Val MSE')
axes[0].set_title(f'traingdx (1-й порядок)\nЭпох: {len(hist_tr2)}, MSE:
{hist_tr2[-1]:.2e}')
axes[0].set_xlabel('Эпоха')
axes[0].set_ylabel('MSE')
axes[0].legend()
axes[0].grid(True)

# Этап 3
axes[1].semilogy(hist_tr3, label='Train MSE')
axes[1].semilogy(hist_vl3, label='Val MSE')
axes[1].set_title(f'trainbfg / LBFGS (2-й порядок)\nЭпох:
{len(hist_tr3)}, MSE: {hist_tr3[-1]:.2e}')
axes[1].set_xlabel('Эпоха')
axes[1].set_ylabel('MSE')
axes[1].legend()
axes[1].grid(True)

plt.suptitle('Сравнение методов обучения', fontsize=14)
plt.tight_layout()
plt.show()

# Сравнение на одном графике
fig, ax = plt.subplots(figsize=(10, 5))

model_gdx.eval()
model_bfg.eval()
T_all_f = torch.FloatTensor(t_func).unsqueeze(1)
with torch.no_grad():
    pred_gdx_all = model_gdx(T_all_f).numpy().flatten()
    pred_bfg_all = model_bfg(T_all_f).numpy().flatten()

ax.plot(t_func, x_func, 'b-', linewidth=2.5, label='Эталон', alpha=0.7)
ax.plot(t_func, pred_gdx_all, 'r--', linewidth=1.5, label='traingdx (1-
й порядок)')
```

```
ax.plot(t_func, pred_bfg_all, 'g-.', linewidth=1.5, label='trainbfg (2-
й порядок)')
ax.set_xlabel('t')
ax.set_ylabel('x')
ax.set_title('Сравнение аппроксимации функции')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()
```

Вывод

В ходе лабораторной работы исследованы свойства многослойной нейронной сети прямого распространения и алгоритмы её обучения.

На первом этапе построена сеть для классификации трёх линейно неразделимых классов (две концентрические окружности и парабола). Сеть с архитектурой [2]→[20, tanh]→[3, sigmoid], обученная алгоритмом RProp, успешно классифицирует точки заданной области. Визуализация с помощью RGB-кодирования выходного вектора демонстрирует нелинейные границы разделения классов.

На втором этапе выполнена аппроксимация функции $x = \cos(t^2 - 2t + 3)$ с помощью метода первого порядка (traingdx — градиентный спуск с моментом и адаптивным шагом). Метод первого порядка сходится медленнее и требует большего числа эпох для достижения приемлемой точности.

На третьем этапе та же функция аппроксимирована с помощью метода второго порядка (trainbfg — квазиньютоновский метод BFGS/LBFGS). Метод второго порядка сходится значительно быстрее и достигает более низкой ошибки за меньшее число эпох, что подтверждает теоретическое преимущество методов второго порядка при обучении нейронных сетей.

Все этапы лабораторной работы выполнены. Графики и численные результаты соответствуют теоретическим ожиданиям.